

U-Net Image Segmentation Report

Muhammad Ahamd
25280065

1. Introduction

U-Net is a convolutional neural network designed for image segmentation. It was introduced in 2015 and is widely used for tasks that require pixel-level classification. The model consists of an encoder and decoder structure with skip connections that help preserve spatial information and improve segmentation accuracy.

2. Dataset Description

The dataset provided for this task consists of:

- RGB images
- Corresponding segmentation masks

Each mask represents pixel-wise annotations of the objects present in the image.

3. Data Preparation

The dataset was split into training and testing sets using:

- **Training Set:** 70%
- **Testing Set:** 30%

This split was performed using `train_test_split` from `sklearn`.

Preprocessing Steps

- Images were read using OpenCV
- Converted from BGR $\rightarrow$ RGB
- Normalized to range $[0, 1]$
- Converted to PyTorch tensors

- Masks were converted to integer labels
- Center cropping applied to size **324 × 324**

4. Model Architecture

The U-Net model was implemented manually using PyTorch.

Key Components:

- Encoder with convolution + max pooling layers
- Bottleneck layer
- Decoder with transpose convolution (upsampling)
- Skip connections using concatenation
- Final 1×1 convolution for class prediction

The model uses:

- **Triple convolution blocks** in encoder/decoder
- **ConvTranspose2d** for upsampling
- **Center cropping** to align feature maps

5. Training Configuration

The model was trained using the following parameters:

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** Adam
- **Learning Rate:** 0.001
- **Epochs:** 50
- **Batch Size:** 8
- **Device:** GPU (if available), otherwise CPU

The training loop included:

- Forward pass
- Loss computation

- Backpropagation
- Weight updates

Choice of Training Hyperparameters

The hyperparameters were selected to ensure stable training and good performance.

- **Learning Rate (0.001):**
A value of 0.001 was used because it provides a good balance between speed and stability. A higher value can make training unstable, while a lower value slows down learning.
- **Optimizer (Adam):**
Adam was chosen because it automatically adjusts the learning rate and helps the model converge faster.
- **Loss Function (CrossEntropyLoss):**
This loss function is suitable for multi-class segmentation, where each pixel belongs to a specific class.
- **Number of Epochs (50):**
The model was trained for 50 epochs to allow enough learning without overfitting.
- **Batch Size (8):**
A batch size of 8 was used due to memory limits and to maintain stable training.

6. Evaluation Metrics

6.1 Mean Intersection over Union (Mean IoU)

Mean IoU was calculated for each image and averaged across the test dataset.

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$

6.2 Pixel Accuracy

Pixel accuracy was also calculated as:

$$\text{Accuracy} = \frac{\text{Correct \setminus Pixels}}{\text{Total \setminus Pixels}}$$

Both metrics were computed for training and testing datasets.

7. Results

7.1 Quantitative Results

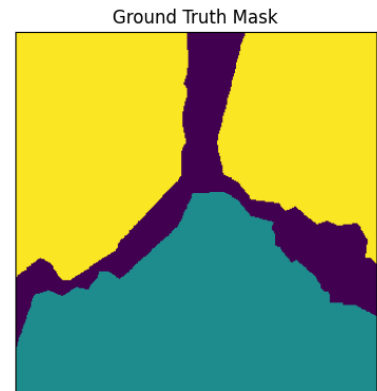
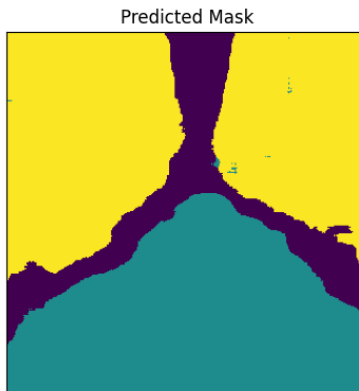
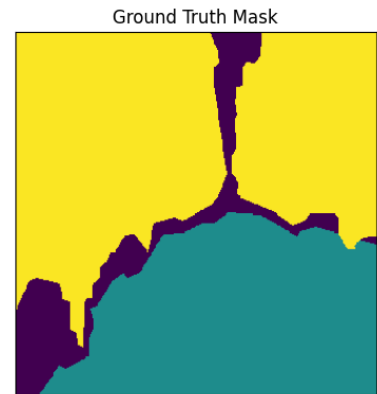
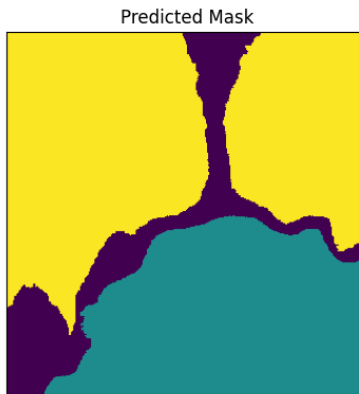
Based on the implementation:

- **Train Accuracy:** 0.9433735280677497
- **Test Accuracy:** 0.9202648879907834
- **Mean IoU:** 0.8135

7.2 Sample Outputs

Five sample test results were generated showing:

- RGB Image
- Predicted Mask
- Ground Truth Mask



RGB



Predicted Mask



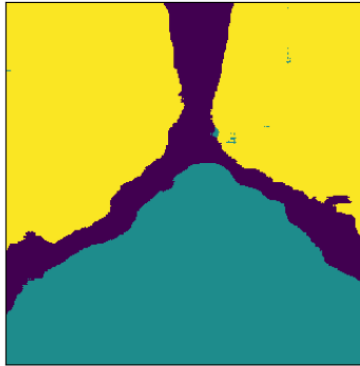
Ground Truth Mask



RGB



Predicted Mask



Ground Truth Mask



RGB



Predicted Mask



Ground Truth Mask

